BiT
Bahir Dar Institute Of Technology
ባሕር ዳር ቴክኖሎጂ ኢንስቲትዩት
Bahir Dar University
ባሕር ዳር ዩኒቨርሲቲ

FACULTY OF COMPUTING

DEPARTMENT OF SOFTWARE ENGINEERING

COURSE TITLE : OPERATING SYSTEM AND SYSTEM PROGRAMMING

OPERATING SYSTEM: DISK OPERATING SYSTEM

Name : Biruh Bizualem

ID Number: BDU1701727

Section :B

Submission Date:April 28,2026

Submitted to:Lec.Wondimu

Contents

1. Introduction **1**

1.1 Background of Operating Systems **1**

1.1.1 Background of DOS 2

1.2 Importance of System Programming **5**

1.3 Motivation for Using DOS in Modern Study **6**

2. Objectives **6**

3. Requirements **7**

3.1 Hardware Requirements **7**

3.2 Software Requirements **9**

4. Installation Steps **12**

4.1 Creating a Virtual Machine **12**

4.2 Configuring Virtual Machine Settings **13**

4.3 Booting from the ISO Installation Media **14**

4.4 Disk Partitioning Using FDISK **15**

4.5 Initializing the File System: **16**

4.6 Installing DOS **17**

4.7 Final Boot **18**

4.8 System Identity and Resource Management: **19**

5. Issues Faced and their Solutions **20**

6. Filesystem Support **23**

7. Advantages and Disadvantages **27**

7.1 Advantages of DOS **27**

7.2 Disadvantages of DOS **29**

8. Conclusion **30**

9. Future Outlook / Recommendation **31**

10. Virtualization Explanation **32**

10.1 What is Virtualization **32**

10.2 Why Virtualization is Used **33**

10.3 How Virtualization Works **34**

11. References **37**

Installation and Analysis of DOS in a Virtual Environment

1. Introduction

1.1 Background of Operating Systems

An operating system (OS) is a critical system software layer that acts as an intermediary between computer hardware and users. It provides an execution environment for applications while managing all hardware resources efficiently and securely.

At a technical level, an OS performs four fundamental functions:

- **Process Management:** Scheduling, creation, and termination of processes
- **Memory Management:** Allocation, tracking, and optimization of RAM usage
- **File System Management:** Organizing and storing data on storage devices
- **Device Management:** Controlling input/output hardware via drivers

Without an operating system, every software program would need to directly control hardware components, which would make computing inefficient, unsafe, and extremely complex.

<u>Evolution of Operating Systems</u>

Operating systems evolved in response to increasing hardware complexity and user demand.

❖ **First Generation (1940s–1950s): No Operating System**

- Programs were written in machine language
- Execution was manual using punch cards or switches
- Only one program executed at a time

Why this stage matters:
It highlights the necessity of automation, which later led to OS development.

❖ **Second Generation (1950s–1960s): Batch Systems**

- Jobs were collected and executed in batches
- No user interaction during execution

Why this matters:
It improved efficiency but lacked interactivity, showing the need for user-controlled systems.

- **Third Generation (1960s–1980s): Multiprogramming and Time-Sharing**

- Multiple users could access a system simultaneously
- CPU time was shared among processes

Why this matters:
This introduced multitasking concepts, which DOS lacked, showing its limitations.

- **Fourth Generation (1980s–Present): Modern Operating Systems**

- Graphical interfaces
- Multitasking and multiprocessing
- Advanced security models
- Networking capabilities

Why this matters:
It demonstrates how far operating systems have evolved beyond DOS.

1.1.1 Background of DOS

Historical Origin of DOS

Disk Operating System (DOS) was developed in the early 1980s when personal computers were becoming commercially viable. At that time:

- Computers had extremely limited processing power
- Memory was measured in kilobytes
- Storage devices were primarily floppy disks
- No standard operating system existed for IBM-compatible PCs

IBM introduced the IBM PC in 1981 and required an operating system. Microsoft developed **MS-DOS**, which became the dominant OS for early personal computing.

Why DOS Was Created

DOS was created to solve three major computing problems:

1. Lack of Hardware Abstraction

Early computers required direct hardware control.

Why this was a problem:
Every program had to be hardware-specific, making software development inefficient and non-portable.

2. Lack of Program Execution Management

There was no system to:

- Load programs into memory
- Execute programs safely
- Manage input/output operations

Why DOS solved this:
It provided a standardized execution environment.

3. Lack of File Organization

Storage devices had no structured system.

Why DOS solved this:
It introduced structured file management using FAT.

Architecture of DOS

DOS uses a **monolithic architecture**, meaning all components are tightly integrated.

1. BIOS (Basic Input/Output System)

- Initializes hardware during boot
- Provides basic device communication

Why BIOS is needed:
Because DOS cannot directly interface with modern hardware without initialization support.

2. DOS Kernel (IO.SYS + MSDOS.SYS)

- Core system layer
- Handles memory, files, and program execution

Why it is important:
It forms the operational heart of DOS.

3. COMMAND.COM (Shell)

- Provides command-line interface
- Executes user commands

Why CLI was used instead of GUI:
Graphical interfaces required more memory and processing power than available at the time.

Execution Model of DOS

DOS is:

- Single-user system
- Single-task system

Only one program executes at a time.

Why DOS is Single-Tasking

- CPUs lacked multitasking support
- Memory was insufficient for multiple processes
- Simplicity ensured stability and performance

Deep Insight:
Multitasking requires process isolation and memory protection, which early hardware could not support.

Memory Management in DOS

DOS uses segmented memory based on Intel 8086 architecture:

- 0–640 KB → Conventional Memory
- 640 KB–1 MB → System Reserved Area
- Above 1 MB → Extended Memory (limited usage)

Why the 640 KB Limit Exists

- Hardware design limitation of 16-bit processors
- Address space constraints of early Intel CPUs

Why this is important academically

- It explains evolution toward virtual memory systems
- Shows early OS design limitations

- Demonstrates need for modern memory management techniques

File System in DOS (FAT System)

DOS uses the **File Allocation Table (FAT)** system.

How FAT Works

- Disk divided into clusters
- Each file stored in linked clusters
- FAT table tracks file locations

Why FAT was used

- Extremely simple design
- Minimal CPU and memory requirements
- Suitable for floppy disks and small drives

Why 8.3 File Naming Exists

- Limited system memory
- Simple directory indexing

Hardware Interaction in DOS

DOS allows direct hardware access without abstraction.

Why this was designed

- Maximized performance
- Reduced system complexity
- Hardware was simple and predictable

Why this is not used today

- No protection between processes
- High risk of system crashes
- Security vulnerabilities

1.2 Importance of System Programming

System programming refers to writing software that directly interacts with hardware and system resources.

Why system programming is important in this project

- DOS exposes low-level system behavior directly
- Helps understand system calls and interrupts
- Demonstrates OS-hardware interaction clearly
- Builds foundation for kernel development and embedded systems

System programming is essential because operating systems are fundamentally control programs for hardware.

1.3 Motivation for Using DOS in Modern Study

Why DOS is still studied today

- Simplest form of operating system
- Provides full visibility of system operations
- Helps understand OS evolution

Why DOS is used in this project

- Lightweight and easy to install in virtual machines
- No modern dependencies required
- Ideal for educational experimentation

Academic Reason:
DOS serves as a foundational model that helps explain how modern operating systems evolved.

2. Objectives

The main objectives of this project are:

- To install DOS in a virtual environment using VMware Workstation and/or VirtualBox
- To understand the installation process of a low-level operating system
- To analyze DOS system architecture and limitations
- To explore file system structures used by DOS
- To identify challenges encountered during virtualization and propose solutions
- To understand the role of virtualization in modern computing

Why these objectives are important:
They ensure both theoretical understanding and practical experience, bridging the gap between conceptual learning and real-world system implementation.

3. Requirements

3.1 Hardware Requirements

To successfully install and analyze DOS in a virtualized environment using VMware Workstation or VirtualBox, the host system must meet certain minimum hardware specifications. Although DOS itself is extremely lightweight, virtualization introduces an additional layer that requires significantly more computing resources.

- **Processor (CPU): Minimum 1 GHz (Multi-core Recommended)**

A modern CPU with at least 1 GHz processing speed is required.

Why this is required:

- Virtualization involves running a complete "virtual computer" inside the host system.
- The hypervisor must allocate CPU cycles not only for the host operating system but also for the guest OS (DOS).
- Even though DOS itself is simple, the virtualization engine performs continuous background computation such as:
    - Hardware emulation
    - Instruction translation
    - Device simulation
- Multi-core CPUs improve performance by allowing parallel execution of host and guest operations.

Without sufficient CPU resources, the virtual machine may experience:

- Slow boot times
- Input lag
- Failure in disk emulation processes

This is because virtualization is computationally expensive despite the simplicity of the guest OS.

- **Random Access Memory (RAM): Minimum 2 GB (Host System Requirement)**

At least 2 GB of RAM is required on the host system.

Why this is required:

- The host OS must allocate memory for:
    - System processes
    - Virtual machine software
    - Guest operating system (DOS environment)
- Virtualization requires memory duplication, meaning RAM is divided between:
    - Host system memory usage
    - Virtual machine allocated memory

Although DOS itself only requires a few megabytes of RAM, virtualization software may consume hundreds of megabytes to simulate:

- Virtual BIOS
- Virtual disk buffers
- Emulated hardware devices

Therefore, insufficient RAM leads to:

- VM crashes
- Severe performance degradation
- Boot failures in extreme cases

➢ **<u>Storage Capacity:</u> Minimum 10 GB Free Disk Space**

A minimum of 10 GB free storage space is required on the host system.

Why this is required:

- Virtual machines are stored as disk image files (e.g., VMDK or VDI).
- These files dynamically grow as the guest OS operates.
- Additional space is needed for:
    - Virtual disk image
    - ISO installation files
    - Snapshots (if enabled)
    - Log files generated by the hypervisor

Even though DOS itself requires only a few megabytes, virtualization introduces storage overhead because:

- Every disk operation is emulated and stored in virtual format
- File system simulation requires buffer storage
- System state snapshots may duplicate entire VM states

Thus, storage is not determined by DOS but by the virtualization environment.

- **<u>Virtualization Support:</u> Intel VT-x or AMD-V (Hardware Virtualization Technology)**

Modern processors must support hardware virtualization extensions such as Intel VT-x or AMD-V.

Why this is required:

- Virtualization relies on hardware-assisted execution of guest instructions
- Without hardware virtualization support, the CPU must use software emulation, which is significantly slower
- VT-x/AMD-V allows:
    - Direct execution of guest instructions
    - Efficient context switching between host and guest
    - Reduced overhead in system emulation

Without hardware virtualization:

- Performance drops drastically
- Some operating systems may fail to boot
- Hardware simulation becomes inefficient and unstable

This makes virtualization either impractical or extremely slow.

3.2 Software Requirements

The software environment is equally critical for successful DOS installation and analysis.

- **<u>Virtualization Software</u>: VMware Workstation or VirtualBox**

A hypervisor such as VMware Workstation or Oracle VirtualBox is required.

Why this is required:

- DOS cannot run natively on modern hardware due to incompatibility with:
    - Modern CPUs
    - UEFI firmware systems
    - SATA/NVMe storage controllers
- Virtualization software emulates a complete hardware environment, including:
    - CPU architecture
    - BIOS/firmware
    - Storage devices
    - Input/output devices

Without virtualization:

- DOS installation would fail on modern systems
- Legacy BIOS-based boot processes would not be supported
- Hardware drivers required by DOS would be unavailable

Thus, virtualization acts as a **compatibility bridge between legacy software and modern hardware**.

- **DOS Installation Media: MS-DOS or FreeDOS ISO Image**

A bootable DOS image is required for installation.

Why this is required:

- DOS is not distributed in modern systems as a pre-installed OS
- ISO image provides:
    - Bootable system environment
    - Installation files (IO.SYS, MSDOS.SYS, COMMAND.COM)
    - Disk utilities like FDISK and FORMAT

The ISO acts as a virtual replacement for physical installation media such as:

- Floppy disks (historically used)
- Bootable CDs

Without this image:

- The virtual machine would not have a bootable operating system
- Installation process cannot begin

➢ **<u>Host Operating System:</u> Windows / Linux / macOS**

A modern host operating system is required to run virtualization software.

Why this is required:

- The host OS manages:
    - Hardware access
    - Virtual machine execution
    - Resource allocation
- It acts as a foundation for running hypervisors

The guest OS (DOS) depends entirely on the host OS for:

- CPU scheduling
- Memory allocation
- Hardware abstraction

Without a host OS:

- Virtual machines cannot function
- Hardware access cannot be virtualized
- DOS cannot be executed in modern systems

3.3 Overall Importance of Requirements (System-Level Insight)

Why These Requirements Matter Collectively

Each requirement is not independent; they work together to ensure:

- Stable virtualization environment
- Accurate emulation of legacy hardware
- Smooth installation and execution of DOS
- Prevention of system incompatibilities

The requirements exist because:

- DOS is a **legacy operating system designed for 1980s hardware**
- Modern computers are fundamentally incompatible with DOS architecture
- Virtualization introduces a **simulation layer that must be supported by real hardware resources**

Therefore, requirements are not based on DOS itself, but on the **complexity of simulating DOS inside a modern computing environment**.

4. Installation Steps

4.1 Creating a Virtual Machine

1. Launch VMware Workstation or Oracle VirtualBox
2. Click on “Create New Virtual Machine”
3. Select installation mode:
    - Typical (VMware) or Standard (VirtualBox) for automatic configuration
4. Choose installation source:
    - Select the DOS ISO file as the bootable media

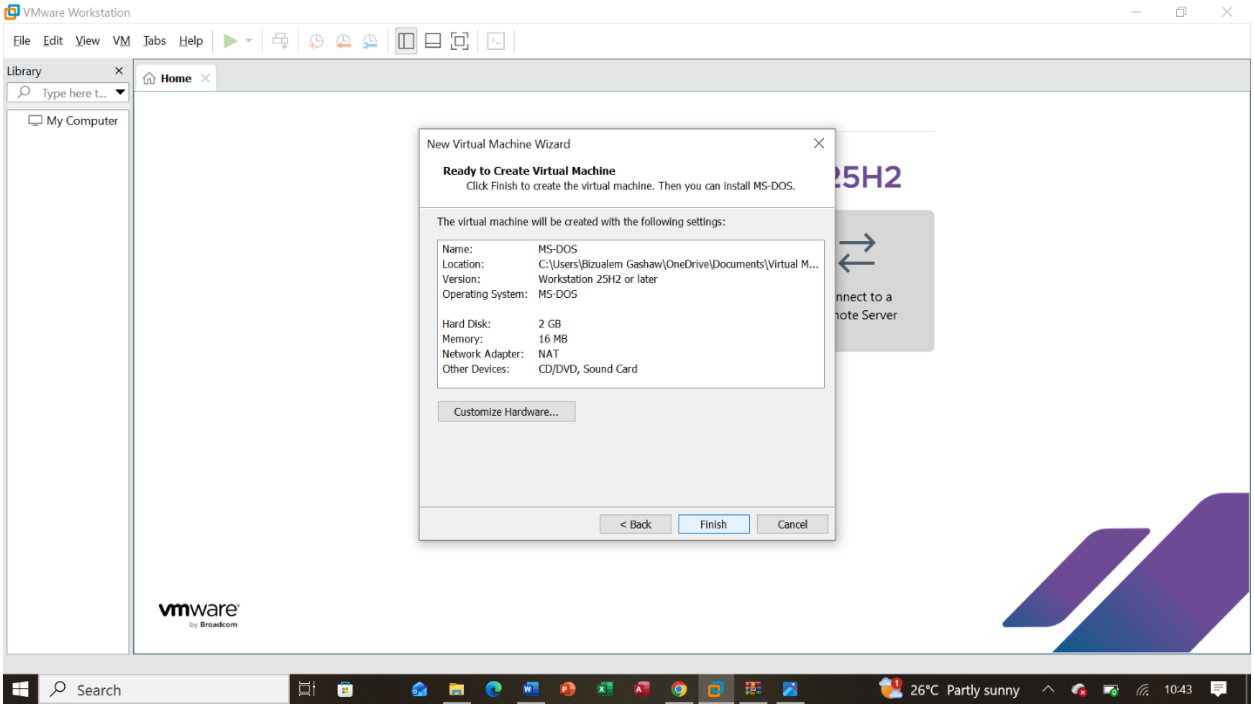

Figure 4.1 VM CREATION INTERFACE

Why this step is necessary:

A virtual machine acts as a software-based simulation of a physical computer system. Instead of installing DOS directly on real hardware (which is incompatible with modern systems), the VM creates a controlled environment that mimics:

- CPU architecture
- BIOS firmware
- Storage devices
- Memory system

DOS was designed for legacy hardware such as BIOS-based IBM PCs. Modern systems use UEFI and advanced hardware controllers that DOS cannot recognize. Therefore, a virtual machine is required to bridge the technological gap between legacy software and modern hardware architectures.

4.2 Configuring Virtual Machine Settings

- Operating System Type: Other / DOS
- RAM: 16 MB – 64 MB
- CPU: 1 core
- Disk Size: 500 MB – 2 GB
- Disk Type: IDE (Highly recommended)

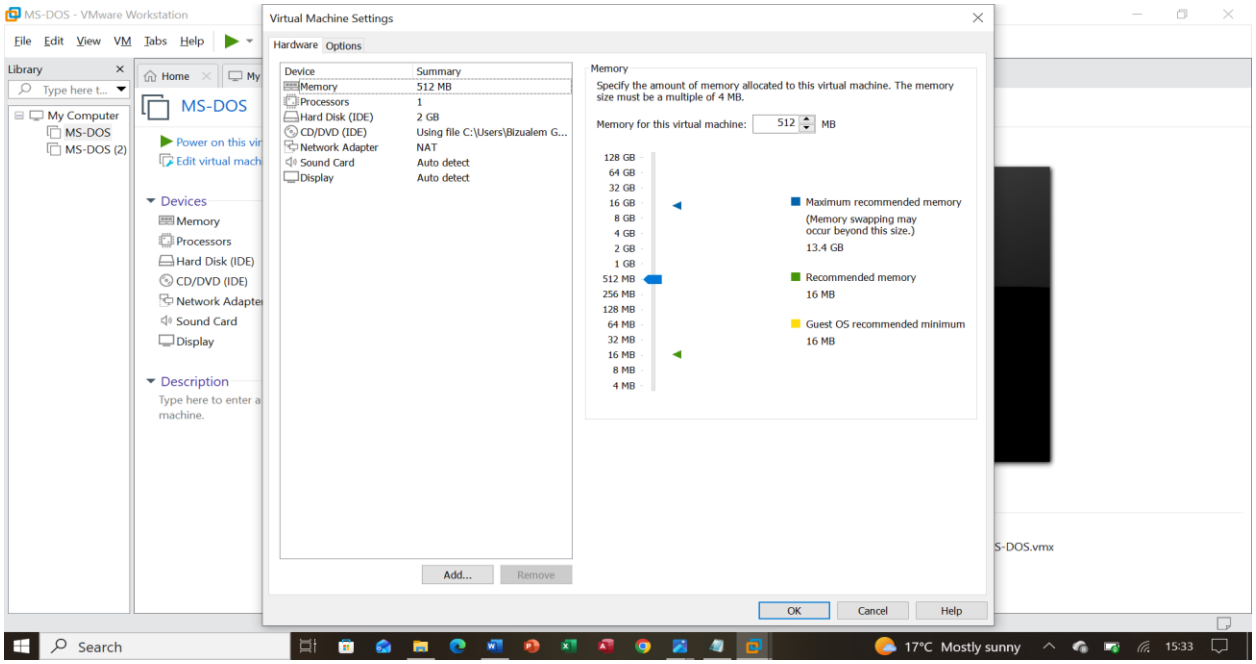

Figure 4.2 VM Configuration Settings

Why these settings are required:

These settings are carefully chosen to replicate a DOS-compatible hardware environment.

- Low RAM allocation (16–64 MB):
  DOS is a 16-bit operating system and cannot address large memory spaces. Allocating excessive RAM does not improve performance and may cause instability in emulation.
- Single CPU core:
  DOS does not support multiprocessing or multithreading. Multiple cores are unnecessary and may cause timing inconsistencies in older software behavior.
- Small disk size (500 MB–2 GB):
  DOS uses FAT16, which has limited partition size support. Larger disks are unnecessary and sometimes incompatible.
- IDE disk type:
  IDE (Integrated Drive Electronics) mimics older hard disk controllers.

Modern storage systems (SATA/NVMe) use advanced communication protocols that DOS cannot interpret. IDE is chosen because it replicates the simplicity of 1980s storage interfaces, ensuring successful booting and disk recognition.

4.3 Booting from the ISO Installation Media

- **Action:** Power on the virtual machine instance.
- **Configuration:** Verify that the CD/DVD (SATA/IDE) settings in VMware point to the FreeDOS ISO and that the Boot Order prioritizes the optical drive over the Hard Drive (HDD).
- **Result:** The FreeDOS bootloader (ISOLINUX) executes, successfully displaying the live environment and installation menu.

Why this step is necessary:

Booting from an ISO (International Organization for Standardization) file is essential because it serves as the "Primary Boot Device" in a pre-OS state. Since the virtual hard disk is currently uninitialized and contains no partition table or Master Boot Record (MBR), the system requires an external source to provide the initial machine code necessary to run the installer.

The Boot Sequence:

Upon startup, the virtual BIOS/UEFI performs a POST (Power-On Self-Test) and then searches for a Boot Signature ($0xAA55$) in the first sector of the available drives. By setting the ISO as the priority, we redirect the CPU to execute the kernel found on the virtual disc. This ISO acts as a RAM-based operating system (Live Environment), allowing us to interact with the hardware and prepare the persistent storage (the virtual HDD) for a permanent DOS installation. Without this temporary environment, the CPU would encounter a "No bootable device found" error and halt.

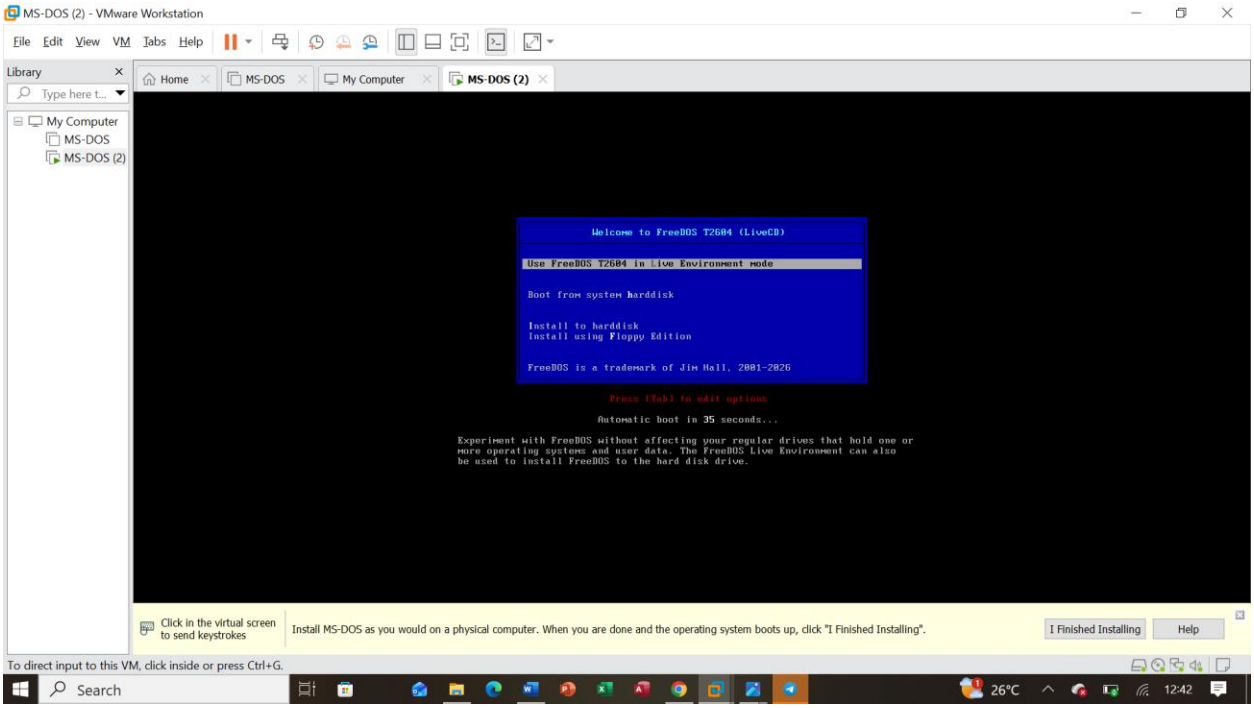

Figure 4.3 *The FreeDOS bootloader interface as seen in VMware. This screen confirms that the virtual machine has successfully mounted the ISO and is ready to execute the installation scripts.*

4.4 Disk Partitioning Using FDISK

1. Run command:
2. FDISK
3. Create a Primary DOS Partition
4. Set the partition as Active (bootable)

Why partitioning is required:

Partitioning defines how the disk is logically structured so that DOS can store and retrieve data efficiently.

DOS relies on a very simple storage model. Without partitions:

- The system would not know where files begin or end
- Boot loaders would not know where the operating system resides
- File system organization would be impossible

Setting a partition as active is essential because it tells the BIOS:

“This is the bootable area containing the operating system.”

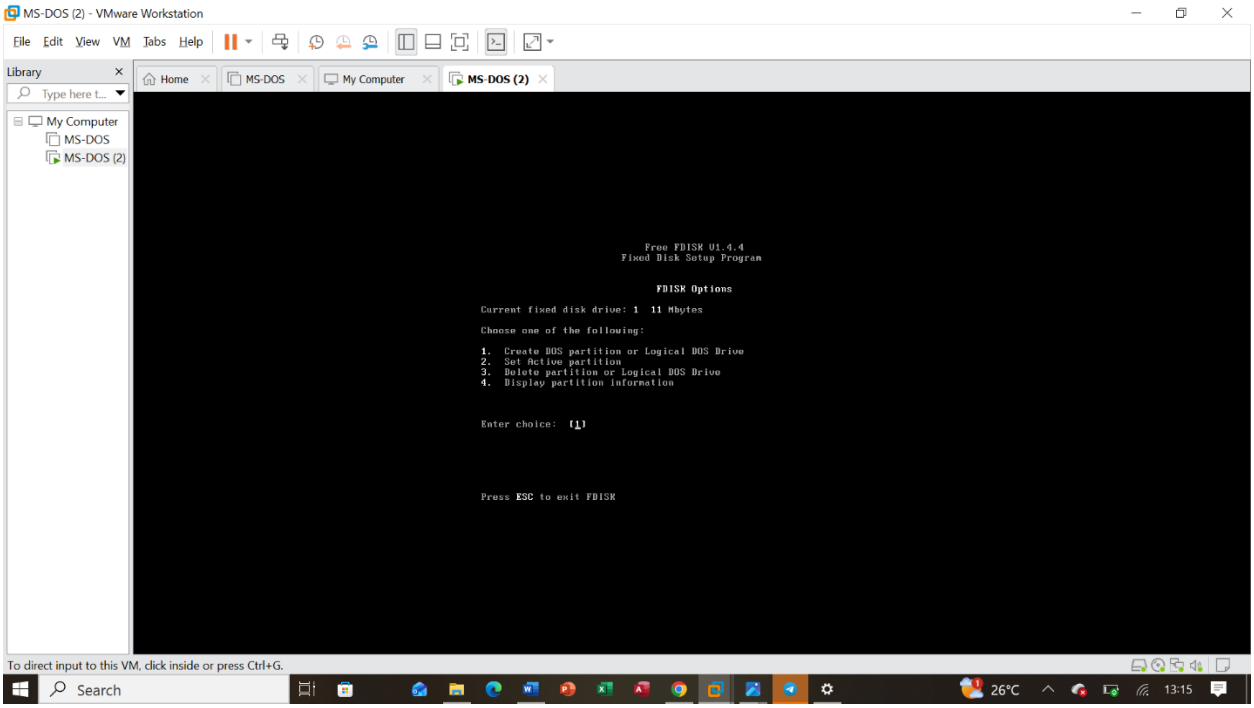

Figure 4.4 FDISK Interface

4.5 Initializing the File System: Formatting Drive C:

- Command Environment: Automatic Installer Utility / FORMAT C:
- File System Type: FAT32 (File Allocation Table, 32-bit)
- Operation: High-level logical formatting of the primary partition.

Why Formatting is Required: Formatting is the process of mapping the physical space of a storage device into a logical structure that the operating system can understand. While partitioning (the previous step) defines the "boundaries" of the drive, formatting creates the "internal architecture." It installs a DOS-compatible FAT file system, which acts as a library index, allowing the system to keep track of where every piece of data is physically located on the disk.

Formatting is Necessary Because:

- **Establishment of Metadata**: A raw disk is merely a collection of magnetic sectors with no organization. Formatting creates the Boot Sector, the File Allocation Tables, and the Root Directory structure.
- **Cluster Management**: As seen in the screenshot, the process defines "allocation units" (clusters). In this instance, the system is configured with 4.0 kbytes per allocation unit, which is the smallest amount of disk space that can be assigned to a file.

- **Logical Mapping**: Without a File Allocation Table, the OS would have no way to "stitch" together files that are stored across non-contiguous sectors. The FAT table provides the "map" that DOS follows to read or write data systematically.

The Foundation of File Organization:

This step essentially creates the foundation of file organization. By "flushing metadata" and "preparing the FAT area," the system ensures that the read/write heads of the virtual drive know exactly where the data boundaries begin and end. Once the Volume Serial Number is generated (as shown in the image), the drive is officially recognized as a logical volume ready for system file installation.

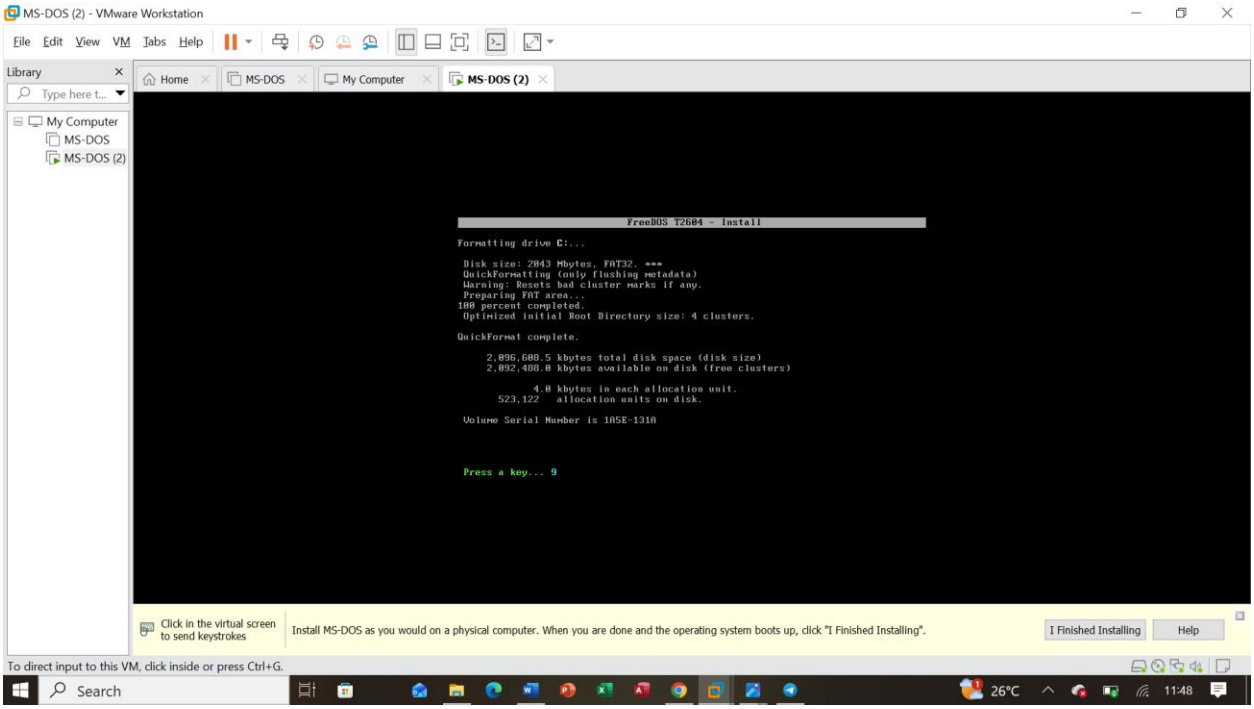

Figure 4.5 *The formatting progress screen within the FreeDOS installer. The output confirms the creation of a 2043 Mbytes FAT32 partition and the successful initialization of 523,122 allocation units.*

4.6 Installing DOS

- Follow on-screen installation prompts
- System files are copied to the disk
- Core files installed:
    - IO.SYS
    - MSDOS.SYS
    - COMMAND.COM

Why this step is required:

This step installs the core operating system components that enable DOS to function.

Without these files:

- The system cannot boot
- No command interface exists
- No file system access is possible

Each file has a specific role:

- IO.SYS → hardware communication
- MSDOS.SYS → kernel operations
- COMMAND.COM → user interface

Together, they form the complete DOS environment.

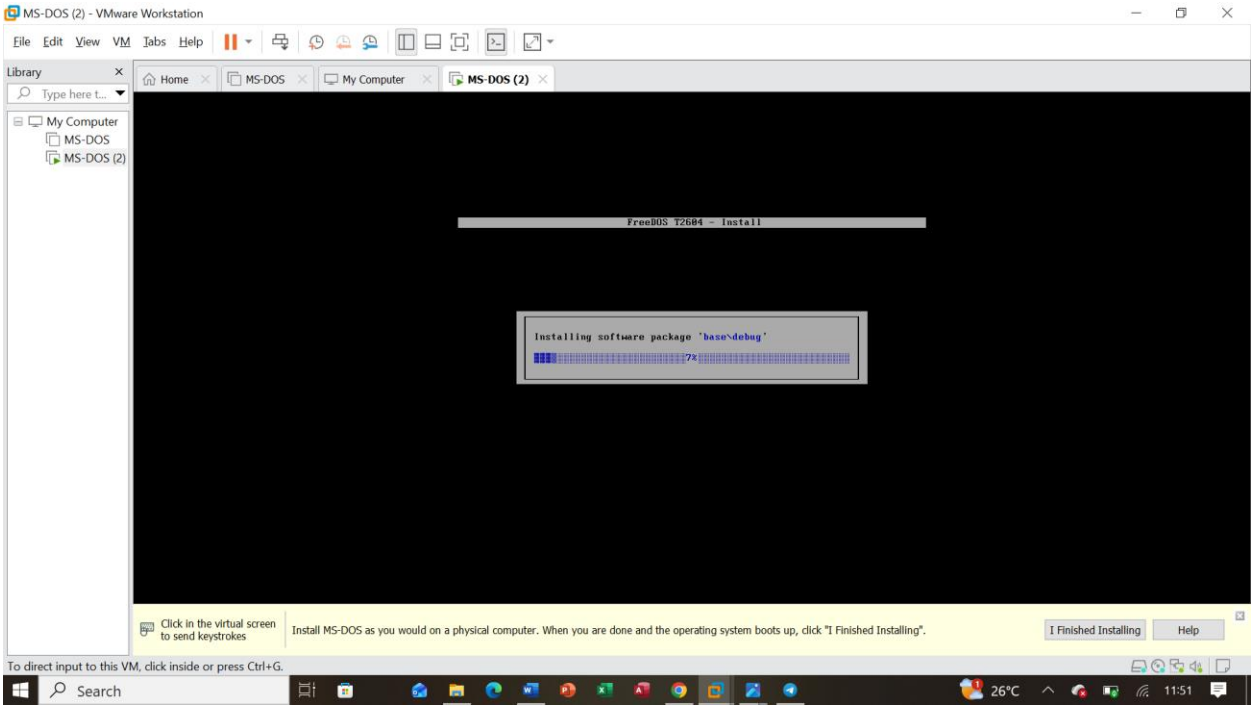

Figure 4.6 Installation Progress

4.7 Final Boot

- **Remove ISO from virtual drive**
- **Restart the virtual machine**

- **DOS prompt appears:**
- **C:\>**

Why this step confirms success:

The appearance of the C:> prompt indicates that:

- The system successfully booted from the hard disk
- The kernel loaded correctly
- File system and command interpreter are functioning

This is the final validation step because:

- **Booting verifies BIOS → bootloader → kernel execution flow**
- **It confirms that disk partitioning, formatting, and installation were successful**
- **It proves that the system is self-bootable without external media**

4.8 System Identity and Resource Management: Single-User Philosophy

- Observation: The system operates directly within a command prompt (C:\>) without requesting credentials.
- Command Analysis: The label and vol commands are used to identify the storage volume itself, rather than a human user.
- Security Model: Null (No authentication layer).

Why DOS Does Not Support User Accounts: Unlike modern multi-user operating systems (like Windows 11 or Linux), DOS was architected as a Single-User, Single-Tasking (SUST) environment. In this model, the operating system does not distinguish between different human operators because the hardware was designed to be used by one person at a desktop at any given time. There is no internal kernel mechanism to track "ownership" of files or to "jail" a user within a specific home directory.

Technical Constraints and Design Philosophy:

- **Hardware and Memory Limitations:** Implementing an authentication subsystem requires memory for a login service and disk space for a user database (like SAM in Windows or /etc/passwd in Linux). In the era of 640KB RAM, every byte was reserved for the application and the basic OS kernel.
- **Processor Architecture:** Early x86 processors (like the 8086/8088) lacked Protected Mode. Without hardware-level memory protection, a "user" could easily bypass any software security by writing directly to system memory.
- **Trusted Environment Assumption:** The computing model assumed physical security. If a person had physical access to the computer, they were the "Administrator" by default. Security was handled by locking the door to the office or removing the bootable floppy disk.

The Proxy for Identity (Volume Labeling): As demonstrated in Figure 4.8, because there are no user accounts, the only way to "personalize" or identify a specific instance of the system is through the Volume Label. By using the label command, a user can name the disk partition (e.g., BIRUH BIZUA). However, as shown in the error message, even this is limited by the legacy 8.3 naming convention logic, restricting labels to 11 characters.

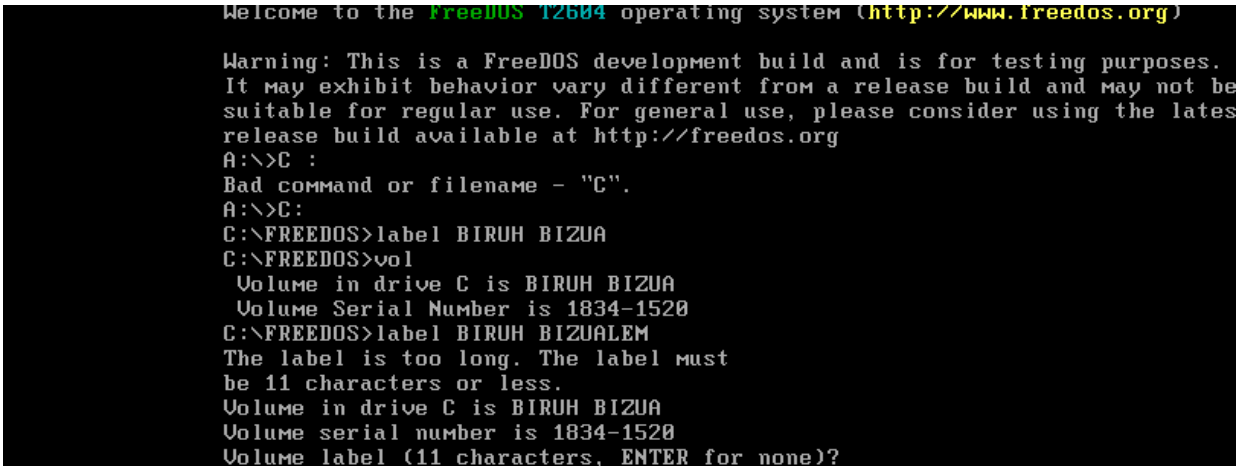

Figure 4.8: *Demonstration of the 'label' and 'vol' commands. In the absence of user account management, the user attempts to identify the system by labeling the Drive C volume. The screenshot also highlights the strict 11-character limit for volume names, a remnant of early FAT file system constraints*

5. Issues Faced and their Solutions

During the installation and execution of DOS in a virtual environment using VMware Workstation or VirtualBox, several technical issues were encountered. These issues primarily arise from the incompatibility between legacy operating systems (DOS) and modern virtualized hardware environments**.**

5.1 VM Not Booting from ISO

Problem Description:

The virtual machine fails to boot from the DOS ISO file, resulting in a blank screen or boot error message**.**

Why this issue occurs:

- The boot order is incorrectly configured, causing the system to skip the CD/DVD drive
- The ISO image may not be properly mounted or recognized by the virtual BIOS
- DOS requires BIOS-based booting, but incorrect VM settings may prioritize the virtual hard disk instead

- Corrupt or non-bootable ISO images may also prevent initialization

Solution Applied:

- Manually set CD/DVD (ISO) as the first boot device in BIOS settings
- Re-mounted the ISO file to ensure proper attachment
- Verified ISO integrity (ensuring it is bootable DOS installation media)
- Reset VM settings to default when configuration conflicts occurred

Why the solution works:

Setting the correct boot priority ensures the system loads the installation media before any operating system exists on the virtual hard disk, which is essential for DOS installation.

5.2 Disk Not Detected

Problem Description:

During installation, DOS fails to detect the virtual hard disk or shows “No fixed disk present”.

Why this issue occurs:

- DOS does not support modern disk controllers such as SATA or NVMe
- Virtual machines may default to incompatible storage controllers
- DOS only recognizes legacy IDE or ATA interfaces
- Missing or incorrectly configured virtual BIOS disk mappings

Solution Applied:

- Changed disk controller type to IDE (Integrated Drive Electronics)
- Recreated the virtual hard disk using DOS-compatible settings
- Ensured the disk was properly attached in VM storage settings

Why the solution works:

IDE emulates older hardware architecture that DOS was originally designed to support, enabling proper disk detection and communication.

5.3 Partition Errors (FDISK Issues)

Problem Description:

Errors occur while creating partitions using FDISK, such as failure to create primary partition or inability to set active partition.

Why this issue occurs:

- Disk not properly initialized or formatted

- Incorrect disk size selection beyond FAT limitations
- Virtual disk corruption or misconfiguration
- DOS requires a single primary partition in most setups

Solution Applied:

- Reinitialized virtual disk and reran FDISK
- Ensured disk size remained within FAT16 limits (≤ 2 GB)
- Created a single primary partition and marked it as active
- Restarted VM after partitioning to apply changes

Why the solution works:

DOS relies on a simple partition structure. Ensuring compatibility with FAT16 constraints allows FDISK to properly define bootable storage regions**.**

5.4 Limited Hardware Compatibility

Problem Description:

DOS fails to recognize modern hardware components such as advanced CPUs, modern chipsets, or virtualized devices.

Why this issue occurs:

- DOS was designed for 16-bit x86 architecture
- It lacks drivers for modern hardware interfaces
- No support for USB, SATA, or advanced memory controllers
- Virtualization introduces abstracted hardware layers that DOS cannot interpret

Solution Applied:

- Reduced VM hardware complexity (single CPU, low RAM)
- Used legacy-compatible settings in VMware/VirtualBox
- Disabled unnecessary virtual devices (sound, USB, network where not needed)

Why the solution works:

Simplifying the virtual hardware environment makes it resemble 1980s–1990s PC architecture, which DOS can successfully operate on.

5.5 Display Issues

Problem Description:

Incorrect screen resolution, distorted text display, or inability to properly render DOS interface.

Why this issue occurs:

- DOS uses text-mode display (typically 80x25 characters)
- Modern GPUs and virtualization display adapters are not natively compatible with DOS graphics standards
- Virtual display drivers may default to high-resolution graphical modes unsupported by DOS

Solution Applied:

- Selected standard VGA or basic display adapter mode in VM settings
- Disabled advanced 3D acceleration features
- Adjusted display scaling within virtualization software

Why the solution works:

DOS relies on simple text-mode output. Using legacy-compatible VGA emulation ensures proper rendering of command-line interface without graphical conflicts.

Overall,All issues encountered in this project arise due to a fundamental reason:

DOS was designed for early physical hardware systems, while it is being executed inside a modern virtualized hardware abstraction layer.

Key Technical Reasons:

- DOS lacks modern hardware drivers
- Virtual machines simulate complex devices that DOS cannot interpret
- File system and partition limitations restrict modern configurations
- BIOS-based booting conflicts with modern UEFI systems (indirectly handled by VM)

Virtualization acts as a translation layer between:

- Modern computing systems (complex, multi-layered, secure)
- Legacy operating systems (simple, direct hardware access)

Therefore, every issue is essentially a compatibility mismatch between simplicity (DOS) and complexity (modern virtualization environments).

6. Filesystem Support

6.1 FAT File Systems

The File Allocation Table (FAT) family of file systems is one of the earliest and simplest disk storage management systems used in personal computing. DOS primarily depends on FAT due to its minimal complexity and compatibility with early hardware architectures.

- ✓ **FAT12 (File Allocation Table 12-bit)**

- Primarily used in floppy disks (360 KB – 1.44 MB)
- Uses 12-bit entries to track cluster allocation
- Very small address space for file storage

Why FAT12 was used:

- Designed for extremely limited storage devices
- Required minimal memory overhead
- Suitable for early portable storage media

FAT12 was sufficient for floppy disks because early systems had:

- Very small file sizes
- Limited processing power
- No requirement for large storage structures

- ✓ **FAT16 (File Allocation Table 16-bit)**

- Main file system used by MS-DOS
- Supports disk sizes up to approximately 2 GB
- Uses 16-bit addressing for cluster management
- Provides simple directory and file allocation structure

Why FAT16 was the primary DOS file system:

- Balanced simplicity and storage capacity
- Compatible with 16-bit DOS architecture
- Efficient for HDDs of the 1980s and early 1990s

FAT16 became standard because it matched the hardware limitations of the time:

- CPUs could handle 16-bit addressing efficiently
- Disk sizes were small compared to modern standards
- File system overhead needed to be minimal

- ✓ **FAT32 (Limited DOS Support)**

- Introduced later for larger storage devices
- Supports significantly larger disk sizes (theoretically up to TB range)
- Uses 32-bit cluster addressing

- Not fully supported by early DOS versions

Why FAT32 has limited DOS compatibility:

- Requires extended system support not available in original DOS
- Designed for Windows 95 OSR2 and later systems
- Increases system complexity beyond DOS design scope

FAT32 represents an evolution beyond DOS capabilities, introducing features that require:

- Larger memory handling
- More advanced file system logic
- Extended boot and driver support

6.2 Comparison with Modern File Systems

Modern file systems are significantly more advanced compared to FAT systems. They introduce features such as security, fault tolerance, and performance optimization.

File System	Key Features	Why It Was Designed
NTFS	**Security permissions, journaling, encryption**	**Designed for Windows to support enterprise-grade security and reliability**
exFAT	**Lightweight, flash optimized, large file support**	**Designed for USB drives and SD cards requiring cross-platform compatibility**
ext4	**Journaling, fast performance, Linux stability**	**Designed for Linux systems requiring reliability and scalability**
ZFS	**Data integrity, self-healing, snapshots**	**Designed for enterprise systems needing high fault tolerance**
HFS+	**macOS legacy file system**	**Designed for older Apple systems with improved storage efficiency**
APFS	**Encryption, snapshots, SSD optimization**	**Designed for modern Apple devices and flash storage efficiency**

Why modern file systems are different from FAT:

Modern file systems were designed to solve problems that FAT cannot handle:

- Data corruption protection
- Large file storage support
- Security and access control
- Efficient disk usage for large drives

- Crash recovery mechanisms

6.3 Why DOS Uses FAT

DOS uses FAT as its primary file system due to fundamental design and hardware constraints.

Reason 1: Simplicity of Design

- FAT uses a very simple table-based structure
- No complex metadata or journaling system
- Easy to implement in low-level environments

Why this matters:

DOS was built for systems with extremely limited processing power, so simplicity was essential for performance and reliability**.**

Reason 2: Low Memory Usage

- FAT requires minimal RAM to operate
- No advanced caching or indexing mechanisms
- Efficient for 16-bit architectures

Why this matters:

Early computers often had less than 640 KB of usable memory, making lightweight file systems necessary.

Reason 3: Hardware Compatibility

- Designed for floppy disks and early hard drives
- Matches BIOS-level disk access methods
- Works with segmented memory architecture

Why this matters:

DOS interacts directly with hardware through BIOS, so the file system must be equally simple and compatible.

Reason 4: Performance Efficiency on Early Systems

- Fast file lookup using simple tables
- Minimal computational overhead
- Suitable for sequential disk access patterns

Why this matters:

Early storage devices were slow and mechanical, so lightweight file systems reduced processing delays.

Why FAT is Important in This Project

Understanding FAT is essential because it:

- Demonstrates how early operating systems managed storage
- Shows the limitations of low-level file system design
- Explains the evolution toward modern file systems like NTFS and ext4
- Helps learners understand the relationship between hardware constraints and OS design

FAT is not just a file system—it is a historical representation of early computing limitations, showing how operating systems were shaped directly by hardware constraints rather than modern performance expectations.

Final Summary

The FAT file system family represents the foundation of early disk management systems. DOS relies on FAT because it aligns with its core design principles:

- Simplicity
- Efficiency
- Low resource consumption
- Hardware compatibility

However, as computing evolved, FAT became insufficient, leading to the development of modern file systems that prioritize security, scalability, and reliability.

7. Advantages and Disadvantages

7.1 Advantages of DOS

1. Lightweight System

DOS has an extremely small system footprint, requiring minimal memory and storage resources.

Why this is an advantage:

- Designed for early computers with very limited RAM (often below 640 KB)
- Does not require background services or graphical interfaces
- Can run efficiently even on very low-spec hardware

Because DOS does not load unnecessary services, almost all system resources are dedicated directly to user commands and program execution, making it extremely efficient in constrained environments.

2. Fast Execution Speed

DOS executes commands and programs quickly due to minimal abstraction layers.

Why this is an advantage:

- Direct hardware access reduces processing overhead
- No multitasking scheduler is involved
- No graphical rendering delays

Unlike modern operating systems, DOS does not waste CPU cycles managing multiple processes, which results in very fast command execution for single tasks.

3. Simple System Design

DOS follows a simple architecture with limited components.

Why this is an advantage:

- Easier to understand and analyze
- Fewer system layers reduce complexity
- Minimal dependencies between system components

This simplicity makes DOS highly valuable in academic environments, as it clearly demonstrates fundamental OS principles without modern complexity.

4. Direct Hardware Access

DOS allows applications to directly interact with hardware devices.

Why this is an advantage:

- Eliminates abstraction layers
- Improves performance for low-level operations
- Useful for system programming and embedded development
- This design reflects early computing philosophy where performance was prioritized over safety and isolation.

7.2 Disadvantages of DOS

1. No Multitasking

DOS can execute only one program at a time.

Why this is a disadvantage:

- CPU and memory management capabilities were limited in early systems
- No process scheduling mechanism exists
- No support for concurrent execution

Impact:
Users cannot run multiple applications simultaneously, reducing productivity.

2. No Security Mechanisms

DOS does not include user authentication or access control.

Why this is a disadvantage:

- Designed for single-user environments
- No concept of user accounts or permissions
- Hardware assumed to be fully trusted

Impact:
Any program can access or modify system files, making the system vulnerable in shared environments.

3. Limited Memory Support

DOS has severe memory limitations (conventional memory up to 640 KB).

Why this is a disadvantage:

- Based on 16-bit architecture constraints
- No virtual memory or advanced memory management
- Cannot efficiently handle large applications

Impact:
Restricts software complexity and system scalability.

4. No Modern Hardware Compatibility

DOS cannot natively support modern hardware devices.

Why this is a disadvantage:

- Lack of drivers for modern CPUs, GPUs, and storage systems
- No support for USB, SATA, or network hardware
- Designed before modern hardware standards existed

Impact:
Requires virtualization or emulation to run on modern systems.

Why This Analysis Matters

This comparison is important because:

- It demonstrates how operating systems evolve based on hardware advancements
- It highlights the trade-off between simplicity and functionality
- It explains why modern operating systems became more complex
- It shows how early design decisions shaped future OS development

DOS represents the "baseline model" of operating systems. Its limitations directly explain why modern systems include multitasking, security, and advanced memory management.

8. Conclusion

This project successfully demonstrated the installation, configuration, and analysis of DOS within a virtual environment using VMware Workstation and/or VirtualBox.

The practical implementation provided hands-on experience in setting up a legacy operating system, performing disk partitioning using FDISK, formatting storage with FAT file systems, and interacting with the system through a command-line interface.

Key Learning Outcomes

- Understanding of Operating System Fundamentals
  The project reinforced core OS concepts such as process management, memory handling, and file system structure.

Why this is important:
These concepts form the foundation of all modern computing systems.

- Insight into Legacy System Design
  DOS demonstrated how early operating systems were designed under strict hardware constraints.

Why this is important:
It helps explain the evolution of modern OS features such as multitasking and security.

- Hands-on Virtualization Experience
  The use of VMware/VirtualBox provided practical exposure to virtual machine configuration and management.

Why this is important:
Virtualization is widely used in modern IT environments for testing, development, and system analysis.

Final Insight

Overall, the project bridges the gap between theoretical knowledge and practical system-level understanding by analyzing a historically significant operating system.

9. Future Outlook / Recommendation

9.1 Role of Virtualization in Modern Computing

Virtualization technology plays a crucial role in modern computing systems by enabling multiple operating systems to run on a single physical machine.

Key Benefits:

- Safe OS experimentation
  Users can install and test operating systems without risking damage to the host system.
- Efficient resource utilization
  Hardware resources such as CPU, memory, and storage are shared efficiently among multiple virtual machines.
- Multi-OS environments
  Developers and researchers can run different operating systems simultaneously for comparison and testing.

Why virtualization is important:

Without virtualization, studying legacy systems like DOS would require outdated physical hardware, which is impractical and expensive.

9.2 Relevance of DOS Today

Although DOS is no longer used in mainstream computing, it still holds importance in several areas.

1. Educational Tool

DOS is widely used in academic environments to teach fundamental operating system concepts.

Why:
Its simplicity allows students to understand OS principles without complexity.

2. Embedded Systems Usage

Some embedded systems still use DOS-like structures due to their simplicity.

Why:
Embedded environments require lightweight and stable systems with minimal overhead.

3. Historical Importance

DOS represents a major milestone in computing history.

Why:
It laid the foundation for modern operating systems such as Windows.

Recommendation

It is strongly recommended that students:

- Use virtualization platforms to explore multiple operating systems
- Study both modern and legacy systems for comparative understanding
- Gain hands-on experience with system-level tools and commands

Why this recommendation matters:
Understanding only modern systems provides an incomplete picture of computing evolution. Studying legacy systems like DOS helps learners understand *why* modern operating systems are designed the way they are.

Final Insight

The combination of virtualization and legacy system study provides a powerful educational approach that strengthens both theoretical knowledge and practical system-level understanding.

10. Virtualization Explanation

10.1 What is Virtualization

Virtualization is the process of creating a software-based virtual version of a physical computing system**,** including hardware resources such as CPU, memory (RAM), storage devices, and input/output interfaces.

It allows multiple operating systems to run independently on a single physical machine by abstracting hardware resources into virtual environments.

In simple terms, virtualization makes a single computer behave like multiple independent computers, each running its own operating system.

Why Virtualization is Important

- Physical computers are limited in number and cost
- Different operating systems require different hardware environments
- Legacy systems like DOS cannot directly run on modern hardware
- Safe experimentation requires isolated environments

Virtualization eliminates dependency on physical hardware differences by introducing a hardware abstraction layer, allowing operating systems to run regardless of the actual machine architecture.

10.2 Why Virtualization is Used

Virtualization is widely used in modern computing systems due to its flexibility, safety, and efficiency.

1. To Run Multiple Operating Systems on One Machine

Virtualization allows multiple operating systems (such as Windows, Linux, and DOS) to run on the same physical computer simultaneously.

Why this is important:

- Reduces the need for multiple physical systems
- Enables comparison of different operating systems
- Improves hardware utilization efficiency

Without virtualization, each operating system would require separate hardware, making system testing and learning expensive and inefficient.

2. To Isolate Environments

Each virtual machine operates independently from the host system and other virtual machines.

Why this is important:

- Prevents system-wide crashes caused by faulty software
- Protects the host system from viruses or misconfigurations
- Allows experimentation without risk

Isolation ensures that any failure inside a virtual machine remains contained, making virtualization a safe sandbox for system development and testing.

3. For Testing and Development

Virtual machines are widely used in software development and system testing environments.

Why this is important:

- Enables testing across multiple operating systems and configurations
- Eliminates need for physical hardware setup for each environment
- Speeds up debugging and system evaluation

Developers can replicate real-world environments instantly, which improves software reliability and reduces development time.

10.3 How Virtualization Works

Virtualization operates through a specialized software layer called a hypervisor (such as VMware Workstation or VirtualBox).

Core Functions of a Hypervisor:

- Allocates system resources (CPU, RAM, storage) to virtual machines
- Simulates hardware components such as BIOS, disk controllers, and network interfaces
- Manages multiple virtual machines simultaneously on one physical system

Working Mechanism:

1. The hypervisor installs on the host operating system or directly on hardware
2. It creates virtual machines with defined resource limits
3. Each virtual machine runs its own operating system independently
4. The hypervisor schedules and distributes hardware resources among all VMs

Why Virtualization Works This Way

- Modern processors support hardware-assisted virtualization (Intel VT-x / AMD-V)
- This allows near-native execution of guest operating systems
- Hardware emulation **ensures compatibility with older systems like DOS**

Virtualization works by translating virtual hardware requests into real hardware operations, allowing outdated operating systems to function on modern systems without modification.

Why Virtualization is Important in This Project

Virtualization is essential in this DOS installation project because:

- DOS is incompatible with modern hardware systems
- Modern computers use UEFI instead of legacy BIOS required by DOS
- Storage and hardware interfaces are not supported by DOS drivers
- Virtual machines recreate a legacy-compatible computing environment

Virtualization acts as a bridge between modern computing architectures and legacy operating systems, enabling historical systems like DOS to be studied and analyzed safely.

Final Remarks

This project goes beyond simple installation of DOS. It represents an academic exploration of how computing systems have evolved over time and how virtualization enables access to legacy environments.

By working with DOS in a virtualized environment, several key insights are achieved:

- Understanding how early operating systems interacted directly with hardware
- Observing the limitations of primitive system architectures
- Learning how modern tools replicate historical computing environments

Why this matters academically:

Studying DOS through virtualization provides a controlled and safe method to analyze the foundations of operating system design. It helps bridge the gap between theoretical concepts and practical implementation.

Conclusion

In conclusion, virtualization plays a critical role in enabling the study of legacy operating systems such as DOS. It allows modern computing systems to emulate older hardware environments, making it possible to install, run, and analyze outdated operating systems that are otherwise incompatible with current technology.

The project demonstrated how virtualization technologies such as VMware Workstation and VirtualBox can recreate a fully functional DOS environment, providing valuable hands-on experience in system configuration, installation procedures, and low-level operating system behavior.

Key Takeaways:

- **Virtualization enables safe and efficient execution of legacy systems**
- **DOS provides a simplified model for understanding fundamental OS concepts**
- **Hypervisors are essential tools in modern system design and experimentation**

- **Studying older systems improves understanding of modern operating system evolution**

Final Insight:

Ultimately, this project highlights that virtualization is not only a technological tool but also an educational bridge. It connects modern computing systems with historical operating system designs, allowing students and researchers to understand the progression of computing from simple command-line systems like DOS to complex, secure, and multitasking modern operating systems.

11. References

1. Abraham Silberschatz, Operating System Concepts, 10th Edition, Wiley, 2018.
    - (Primary reference for OS fundamentals: process management, memory, file systems)
2. Andrew S. Tanenbaum, Modern Operating Systems, 4th Edition, Pearson, 2015.
    - (Used for OS evolution, architecture, and system design concepts)
3. William Stallings, Operating Systems: Internals and Design Principles, 9th Edition, Pearson, 2018.
    - (Reference for system programming and OS structure)
4. Microsoft Documentation – MS-DOS Overview
    - https://learn.microsoft.com
    - (Used for DOS architecture, history, and system components)
5. MS-DOS Official Historical Documentation
    - (Reference for DOS kernel, COMMAND.COM, and file system behavior)
6. FreeDOS Project Documentation
    - https://www.freedos.org
    - (Used for installation steps, ISO booting, and DOS commands)
7. Oracle Corporation, VirtualBox User Manual
    - https://www.virtualbox.org/manual
    - (Used for virtualization setup and VM configuration)
8. VMware, VMware Workstation Documentation
    - https://docs.vmware.com
    - (Reference for VM creation and hardware configuration)
9. Brian W. Kernighan & Dennis Ritchie, The C Programming Language, 2nd Edition, Prentice Hall.
    - (Supporting reference for system-level programming concepts)
10. GeeksforGeeks
    - https://www.geeksforgeeks.org
    - (Used for simplified explanations of FAT file system, DOS concepts, and OS basics)
11. TutorialsPoint
    - https://www.tutorialspoint.com

- (Used for DOS commands, FDISK, and formatting explanations)

12. IBM Documentation on Early Personal Computers
    - (Reference for historical context of DOS and IBM PC development)